FECAP – Fundação Escola de Comércio Álvares Penteado
Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas
Projeto Interdisciplinar – 2º semestre de 2026

João Pedro Teixeira dos Santos
Daniel Bandini Grande
Raphael Carmo Mendes
Guilherme Koide Rangel

KFKA – PLATAFORMA DE ACOMPANHAMENTO ESCOLAR

Relatório final do projeto

São Paulo
2026

João Pedro Teixeira dos Santos
Daniel Bandini Grande
Raphael Carmo Mendes

KFKA – PLATAFORMA DE ACOMPANHAMENTO ESCOLAR

Relatório final do projeto

Relatório final apresentado ao Projeto Interdisciplinar do curso de Análise e Desenvolvimento de Sistemas da FECAP, integrando as disciplinas de Desenvolvimento Web Full Stack, Banco de Dados, Programação Orientada a Objetos, Estrutura de Dados e Design de Interface Digital.

São Paulo
2026

Resumo

Este relatório descreve o desenvolvimento da **KFKA – Plataforma de Acompanhamento Escolar**, um sistema web responsivo que organiza a comunicação entre a escola, os professores e os pais ou responsáveis de alunos do Ensino Fundamental. O sistema apoia o acompanhamento acadêmico bimestral: o professor registra, para cada aluno e disciplina, uma descrição do desenvolvimento, a média e várias tags; a administração revisa, altera quando necessário, devolve ou publica; e o responsável consulta o relatório publicado, registra ciência, envia uma observação e baixa o relatório em PDF. A administração conta ainda com cadastros completos, vínculos acadêmicos, configuração dos quatro bimestres, histórico de auditoria, relatórios gerenciais e exportação para Excel.

A solução foi construída com **Node.js e Express** na API, **MySQL** no banco de dados e **React com Vite** na interface, com autenticação por **JWT**, senhas protegidas com **bcrypt**, geração de PDF com **PDFKit** e planilhas com **ExcelJS**. O backend foi organizado em camadas e orientado a objetos, com uma classe principal (`SistemaEscolar`) que orquestra a conexão com o banco, os middlewares e as rotas. O sistema foi publicado na plataforma de nuvem **Railway**, com dados fictícios para demonstração, e está acessível em https://projeto12-production.up.railway.app.

Os doze requisitos funcionais do enunciado foram implementados, e o cenário de ponta a ponta definido como critério de "sistema funcional" foi executado com sucesso tanto no ambiente local quanto no ambiente publicado. O documento apresenta o contexto e os objetivos, a organização do trabalho, os requisitos, a arquitetura, o modelo de dados, a construção do backend e do frontend, as medidas de segurança e privacidade, os testes, a implantação, as dificuldades enfrentadas, as limitações e as lições aprendidas.

Palavras-chave: acompanhamento escolar; sistema web; Node.js; React; MySQL; fluxo editorial; LGPD.

Lista de siglas

- **API** – Application Programming Interface (interface de programação de aplicações).
- **CORS** – Cross-Origin Resource Sharing (compartilhamento de recursos entre origens).
- **CRUD** – Create, Read, Update, Delete (criar, ler, alterar e excluir).
- **CSRF** – Cross-Site Request Forgery (falsificação de requisição entre sites).
- **CSV** – Comma-Separated Values (valores separados por vírgula).
- **HTTP** – Hypertext Transfer Protocol.
- **JSON** – JavaScript Object Notation.
- **JWT** – JSON Web Token.
- **LGPD** – Lei Geral de Proteção de Dados Pessoais (Lei nº 13.709/2018).
- **MVP** – Minimum Viable Product (produto mínimo viável).
- **PDF** – Portable Document Format.
- **POO** – Programação Orientada a Objetos.
- **RF** – Requisito funcional.
- **RN** – Regra de negócio.
- **RNF** – Requisito não funcional.
- **SPA** – Single Page Application (aplicação de página única).
- **SQL** – Structured Query Language.
- **XSS** – Cross-Site Scripting.

Sumário

Resumo 3
Lista de siglas 4
Sumário 5
1. Introdução 9
1.1 Contexto 9
1.2 Problema 9
1.3 Objetivo geral 9
1.4 Objetivos específicos 9
1.5 Justificativa 10
1.6 Escopo 10
1.7 Estrutura do relatório 10
2. Equipe, metodologia e organização do trabalho 12
2.1 Integrantes 12
2.2 Metodologia 12
2.3 Ferramentas de trabalho 12
2.4 Linha do tempo do projeto 13
2.5 Uso de inteligência artificial 14
3. Descoberta e requisitos 15
3.1 Usuários do sistema 15
Administrador (secretaria ou coordenação) 15
Professor 15
Pai ou responsável 15
3.2 Requisitos funcionais e como foram atendidos 15
RF01 – Autenticação e perfis 15
RF02 – Cadastros escolares 16
RF03 – Vínculos da turma 16
RF04 – Períodos bimestrais 16
RF05 – Acompanhamento 16
RF06 – Fluxo editorial 16
RF07 – Histórico e auditoria 17
RF08 – Consulta do responsável 17
RF09 – PDF 17
RF10 – Relatórios gerenciais 17
RF11 – Exportação 17
RF12 – Comunicação e retorno 17

3.3 Regras de negócio 18
3.4 Requisitos não funcionais 18
3.5 Backlog e critérios de aceitação 19
4. Tecnologias e ambiente de desenvolvimento 20
4.1 Escolha da tecnologia 20
4.2 Backend 20
4.3 Frontend 20
4.4 Banco de dados 21
4.5 Ambiente e ferramentas 21
5. Arquitetura do sistema 22
5.1 Visão geral 22
5.2 Camadas do backend 22
5.3 A classe principal SistemaEscolar 23
5.4 Conceitos de orientação a objetos aplicados 23
5.5 Estrutura de pastas 24
5.6 Caminho de uma requisição 24
6. Banco de dados 25
6.1 Modelo conceitual 25
6.2 Tabelas 25
6.3 Integridade dos dados 26
6.4 Índices 26
6.5 Auditoria 27
6.6 Carga de dados fictícios 27
6.7 Fuso horário 28
7. Construção do backend (API) 29
7.1 Padrões gerais da API 29
7.2 Autenticação e autorização 29
7.3 Recuperação e troca de senha 30
7.4 Cadastros 30
7.5 Vínculos 30
7.6 Bimestres 31
7.7 Lançamentos do professor 31
7.8 Fluxo editorial e revisão 31
7.9 Área do responsável 32
7.10 Geração de PDF 32
7.11 Relatórios gerenciais e exportação 32
8. Construção do frontend 33

8.1 Organização 33
8.2 Comunicação com a API 33
8.3 Autenticação e rotas protegidas 33
8.4 Hook de carregamento 33
8.5 Componentes reutilizáveis 34
8.6 Telas do módulo Administrador 34
8.7 Telas do módulo Professor 35
8.8 Telas do módulo Pai/Responsável 35
8.9 Telas abertas e página inicial 35
8.10 Design e guia de estilo 35
8.11 Responsividade e acessibilidade 36
9. Segurança e privacidade (LGPD) 37
9.1 Medidas de segurança 37
9.2 Privacidade e LGPD 38
10. Testes e validação 39
10.1 Estratégia 39
10.2 Casos de teste 39
10.3 Exemplos de resultados 40
10.4 Defeitos encontrados e corrigidos 41
10.5 Critério de sistema funcional 41
11. Implantação em nuvem 42
11.1 Plataforma 42
11.2 Arquitetura da implantação 42
11.3 Preparação do código 42
11.4 Variáveis de ambiente 42
11.5 Passo a passo realizado 42
12. Relação com as disciplinas do semestre 44
12.1 Desenvolvimento Web Full Stack 44
12.2 Banco de Dados 44
12.3 Programação Orientada a Objetos 44
12.4 Estrutura de Dados 44
12.5 Design de Interface Digital 45
13. Dificuldades enfrentadas e soluções 46
13.1 Ambiente restrito no laboratório 46
13.2 Troca de máquina com entrega urgente 46
13.3 Reorganização acidental do repositório 46
13.4 Erros recorrentes de iniciante 46

13.5 Erro 500 no cadastro de professores 47
13.6 Falha de segurança nas rotas de usuários 47
13.7 Implantação 47
13.8 Prazo 47
14. Resultados 48
15. Limitações e trabalhos futuros 49
15.1 Limitações conhecidas 49
15.2 Trabalhos futuros 49
16. Lições aprendidas 51
17. Conclusão 52
Referências 53
Apêndice A – Acessos de demonstração 54
Roteiro sugerido de demonstração 54
Apêndice B – Rotas da API 55
Autenticação 55
Cadastros (administrador) 55
Vínculos (administrador) 55
Bimestres 55
Professor 55
Revisão (administrador) 55
Responsável 55
Relatórios (administrador) 55
Apêndice C – Instalação e execução local 56
Apêndice D – Glossário 57

1. Introdução

1.1 Contexto

Em muitas escolas de Ensino Fundamental, o acompanhamento do desempenho dos alunos ainda depende de cadernetas, planilhas soltas, bilhetes e reuniões presenciais. O professor anota suas observações em um lugar, a coordenação confere em outro, e a família só fica sabendo do desenvolvimento do filho no boletim ou em uma reunião de pais. Esse caminho fragmentado gera retrabalho, atrasos, informações divergentes e pouca transparência para os responsáveis.

Ao mesmo tempo, a comunicação escolar precisa ser oficial e cuidadosa: o que chega à família deve ter passado pela revisão da escola, e os dados envolvem crianças e adolescentes, o que exige atenção especial à privacidade, conforme a Lei Geral de Proteção de Dados Pessoais (LGPD).

1.2 Problema

O problema tratado pelo projeto é a ausência de um fluxo único, rastreável e seguro para registrar, revisar, publicar e consultar o acompanhamento bimestral dos alunos. Sem esse fluxo, não há garantia de que o professor registrou dentro do prazo, de que a escola revisou antes da publicação, de que cada família vê apenas os seus filhos, nem de que as alterações feitas pela administração ficam documentadas.

1.3 Objetivo geral

Desenvolver uma aplicação web responsiva, segura e integrada que permita registrar, revisar, publicar e acompanhar informações acadêmicas bimestrais de alunos do Ensino Fundamental, fortalecendo a comunicação entre escola, professores e responsáveis.

1.4 Objetivos específicos

- Implementar autenticação e controle de acesso para administrador, professor e pai ou responsável.
- Estruturar os cadastros de alunos, responsáveis, professores, áreas, disciplinas, turmas e tags.
- Permitir a configuração dos quatro bimestres e de seus períodos de digitação.

- Disponibilizar ao professor o registro de descrição, média e múltiplas tags por aluno, disciplina e bimestre.
- Implementar o fluxo de rascunho, envio, revisão, alteração, devolução e publicação das anotações.
- Oferecer relatórios gerenciais, geração de PDF e exportação de dados compatível com Excel.
- Aplicar conceitos de Estrutura de Dados, Desenvolvimento Web Full Stack, Design de Interface Digital, Banco de Dados e Programação Orientada a Objetos em uma solução única e coerente.

1.5 Justificativa

Uma plataforma única reduz o retrabalho do professor, dá à coordenação controle sobre o que é publicado e oferece à família um canal claro, acessível pelo celular, para acompanhar o filho a cada bimestre. Para a formação da equipe, o projeto reúne em um só produto o conteúdo de cinco disciplinas do semestre, exigindo decisões reais de arquitetura, modelagem, segurança, usabilidade e implantação em nuvem.

1.6 Escopo

O escopo entregue cobre os três módulos de usuário (Administrador, Professor e Pai/Responsável), os doze requisitos funcionais do enunciado (RF01 a RF12), uma página inicial pública de apresentação do sistema e a implantação em nuvem com dados fictícios. Ficaram fora do escopo o envio real de e-mails (a recuperação de senha gera o link, mas o envio é simulado no servidor), a extensão opcional de dashboard e agenda de reuniões, e a documentação interativa da API em Swagger. Esses pontos estão detalhados no capítulo de limitações.

1.7 Estrutura do relatório

O capítulo 2 apresenta a equipe e a forma de trabalho. O capítulo 3 trata dos requisitos e das regras de negócio. O capítulo 4 descreve as tecnologias e o ambiente. Os capítulos 5 a 8 detalham a arquitetura, o banco de dados, o backend e o frontend. O capítulo 9 aborda segurança e LGPD, o capítulo 10 os testes e o capítulo 11 a implantação. O capítulo 12 relaciona o projeto às disciplinas, o capítulo 13 registra as dificuldades enfrentadas, e os capítulos 14 a 17 trazem resultados, limitações, lições

aprendidas e conclusão. Os apêndices reúnem acessos de demonstração, rotas da API, instruções de instalação e um glossário.

2. Equipe, metodologia e organização do trabalho

2.1 Integrantes

O grupo KFKA é formado por **João Pedro Teixeira dos Santos**, **Daniel Bandini Grande** e **Raphael Carmo Mendes**. Um quarto integrante participou do início do semestre e deixou o grupo antes das entregas finais.

João Pedro concentrou o desenvolvimento do backend, do frontend, do banco de dados e da implantação, além da documentação técnica. Daniel e Raphael contribuíram com o repositório, com as entregas das disciplinas e com o design das interfaces, os protótipos e o design system no Figma, os diagramas e a revisão dos documentos. Todos os integrantes possuem commits no repositório do grupo.

2.2 Metodologia

O trabalho seguiu um modelo **incremental e iterativo**, inspirado em práticas ágeis. O cronograma inicial previa seis fases, na ordem Descoberta, Planejamento, Modelagem, Construção, Testes e Implantação. Na prática, a equipe decidiu antecipar a construção: depois de fechar a tecnologia e entender o enunciado, partiu para o código, deixando parte da documentação formal e dos protótipos para depois. Essa escolha permitiu validar cedo as partes mais arriscadas (ambiente, banco, autenticação e arquitetura), mas exigiu esforço concentrado de documentação no final.

A construção foi dividida em **blocos pequenos e testáveis**. Cada bloco tinha um objetivo claro (por exemplo, "cadastro de professores" ou "tela de revisão"), era implementado arquivo por arquivo, testado no Postman ou no navegador e, só então, registrado em um commit. A regra "commitar logo após cada trecho que funciona" nasceu de um incidente de troca de máquina e passou a ser seguida em todo o projeto.

2.3 Ferramentas de trabalho

- **Visual Studio Code** como editor de código, com o terminal integrado (PowerShell).
- **Git e GitHub** para versionamento, no repositório da organização da turma (2026-2-NADS2/Projeto12).

- **MySQL Workbench** e o cliente de linha de comando do MySQL para modelagem e consultas.
- **Postman** (versão desktop) para testar a API.
- **Figma** para protótipos e design system.
- **Railway** para a hospedagem em nuvem.
- **Claude (Anthropic)** como assistente de inteligência artificial, conforme a declaração de uso de IA entregue junto ao projeto.

2.4 Linha do tempo do projeto

O repositório foi criado em 21 de agosto de 2026 e recebeu as pastas por disciplina em setembro. O desenvolvimento do sistema ocorreu principalmente entre o fim de setembro e o início de novembro, e a versão final foi publicada em 9 de novembro de 2026. Até a data deste relatório, o histórico do repositório soma mais de 100 commits. As principais etapas foram:

1. **Planejamento:** leitura do enunciado, cronograma em seis fases, escolha da tecnologia (Node.js com Express, MySQL e React).
2. **Ambiente:** instalação do Node.js, do Git, do MySQL em versão portátil (as máquinas da faculdade não permitem instalação com administrador) e do Postman.
3. **Repositório:** clonagem do repositório do grupo, criação das pastas backend e frontend, primeiro commit e resolução do primeiro conflito de push.
4. **Base do sistema:** servidor Express respondendo, conexão com o MySQL e tela React buscando dados da API, validando a integração das três camadas.
5. **Banco de dados:** tabelas de usuários e alunos e entrega do documento de índices para a disciplina de Banco de Dados.
6. **Orientação a objetos:** reorganização do backend em classes, com a classe principal `SistemaEscolar`, entregue para a disciplina de POO.
7. **Autenticação:** cadastro com hash bcrypt, login com JWT e middlewares de proteção por perfil.
8. **Cadastros e vínculos:** alunos, professores, responsáveis, áreas, disciplinas, turmas, tags e seus vínculos.
9. **Bimestres, acompanhamento e fluxo editorial:** períodos de digitação, lançamentos do professor, revisão, devolução, publicação e histórico.

10. **Área do responsável, PDF, relatórios e Excel:** consulta, ciência, observação, PDF, indicadores e exportação.
11. **Frontend completo:** telas dos três módulos com base no guia de estilo e nos desenhos do Figma.
12. **Testes, implantação e documentação:** plano de testes, backlog, declaração de uso de IA, publicação no Railway, página inicial pública e este relatório.

2.5 Uso de inteligência artificial

A equipe utilizou o assistente Claude como apoio durante o projeto: para explicar conceitos, propor a estrutura do código, sugerir trechos de implementação, ajudar a diagnosticar erros e apoiar a redação de documentos. O trabalho foi conduzido de forma guiada: o assistente explicava cada passo, e o integrante responsável digitava ou colava o código, executava, testava e fazia os commits, aprendendo os conceitos ao longo do processo. As decisões de escopo, os testes, a validação do funcionamento e a publicação foram feitos pela equipe. O uso está registrado com mais detalhes no documento "Declaração de Uso de IA", na pasta Documentos/Documentação do repositório, e de forma resumida no README.

3. Descoberta e requisitos

3.1 Usuários do sistema

O sistema atende três perfis, cada um com permissões próprias, seguindo o princípio do menor privilégio.

Administrador (secretaria ou coordenação)

Responsável pela estrutura da escola e pela qualidade do que é publicado. Precisa cadastrar pessoas e turmas, definir os prazos dos bimestres, revisar os registros dos professores, publicá-los ou devolvê-los e extrair relatórios. É o único perfil que cria contas de acesso.

Professor

Registra o acompanhamento dos alunos nas turmas e disciplinas que leciona. Precisa de uma tela rápida para lançar descrição, média e tags, salvar rascunhos, enviar para revisão, corrigir o que foi devolvido e ver os retornos das famílias. Só enxerga as suas turmas e só lança dentro do período aberto.

Pai ou responsável

Acompanha um ou mais filhos. Precisa ver, de forma simples e pelo celular, os relatórios publicados por bimestre e disciplina, registrar que tomou ciência, enviar uma observação à escola e baixar o relatório em PDF. Só vê os alunos aos quais está vinculado e apenas o que já foi publicado.

3.2 Requisitos funcionais e como foram atendidos

O enunciado definiu doze requisitos funcionais. Todos foram implementados. A seguir, cada requisito é descrito junto com a forma como foi atendido no sistema.

RF01 – Autenticação e perfis

Login com e-mail e senha, emissão de token JWT válido por 8 horas, logout no frontend (descarte do token), recuperação de senha por link temporário de 30 minutos, troca de senha pelo próprio usuário e autorização por perfil em todas as rotas. Após o login, cada perfil é levado à sua área (/admin, /professor ou /responsavel), e as rotas de um perfil são bloqueadas para os demais.

RF02 – Cadastros escolares

O administrador cadastra, consulta, edita e inativa alunos, professores, responsáveis, áreas de disciplina, disciplinas, turmas e tags, em uma única tela de Cadastros organizada em sete abas. As listas maiores são paginadas e têm busca. A inativação é lógica: o registro sai das listas, mas o histórico é preservado.

RF03 – Vínculos da turma

Na tela de cada turma, o administrador matricula alunos e aloca disciplinas com o professor responsável. O sistema impede vínculos duplicados (o mesmo aluno duas vezes na turma, a mesma disciplina duas vezes) e incompatíveis (um aluno em duas turmas no mesmo ano letivo). Na tela de cada responsável, o administrador vincula os alunos que ele pode acompanhar, com o parentesco; um responsável pode ter vários alunos, e um aluno pode ter vários responsáveis.

RF04 – Períodos bimestrais

O administrador define a data e a hora de abertura e de encerramento da digitação de cada um dos quatro bimestres do ano letivo. O sistema valida que o encerramento seja posterior à abertura e que os períodos não se sobreponham, e calcula a situação de cada bimestre (aguardando, aberto ou encerrado) com base no horário de Brasília.

RF05 – Acompanhamento

O professor escolhe a turma, a disciplina e o bimestre, vê a lista de alunos com a situação de cada registro e abre o formulário de lançamento, onde informa a descrição (até 2000 caracteres), a média (de 0 a 10, com no máximo uma casa decimal) e uma ou mais tags. Cada acompanhamento é único para a combinação de aluno, turma, disciplina e bimestre.

RF06 – Fluxo editorial

O registro nasce como **rascunho**, pode ser editado e é **enviado** pelo professor. O administrador abre a fila de revisão, pode alterar o conteúdo, **devolver** com uma observação para ajustes ou **publicar**. O registro devolvido volta ao professor, que corrige e reenvia. Apenas registros publicados chegam ao responsável.

RF07 – Histórico e auditoria

Toda mudança relevante (criação, edição, envio, alteração administrativa, devolução e publicação) gera um registro no histórico, com o usuário, a data e a hora, o estado anterior e o estado posterior. Na tela de revisão, o administrador vê esse histórico como uma linha do tempo. O acompanhamento também guarda quem revisou e as datas de envio, revisão e publicação.

RF08 – Consulta do responsável

O responsável vê apenas os alunos vinculados a ele e, para cada aluno, os relatórios publicados, organizados por bimestre e disciplina, com média, tags e descrição. A tentativa de acessar um aluno não vinculado retorna "não encontrado", sem revelar se o aluno existe.

RF09 – PDF

O responsável baixa, com um clique, o relatório do bimestre em PDF, gerado no servidor com a biblioteca PDFKit. O arquivo traz a identificação do aluno, da turma e do bimestre e, para cada disciplina, a média, as tags e a descrição publicada.

RF10 – Relatórios gerenciais

O administrador consulta indicadores por bimestre: médias por turma e disciplina, situação dos acompanhamentos (quantos estão em rascunho, enviados, devolvidos e publicados), frequência de uso de cada tag e retornos das famílias. O professor consulta os registros das suas turmas e os retornos dos responsáveis.

RF11 – Exportação

O administrador exporta os dados de acompanhamento em formato compatível com Excel, tanto em planilha .xlsx (gerada com a biblioteca ExcelJS) quanto em arquivo .csv.

RF12 – Comunicação e retorno

O responsável registra ciência do relatório de cada bimestre e pode enviar uma observação. A ciência é única por bimestre (uma segunda tentativa é recusada). A observação fica visível para o professor, na tela de retornos, e para o administrador, nos relatórios.

3.3 Regras de negócio

As regras de negócio do enunciado foram implementadas no servidor, de modo que não podem ser contornadas pela interface ou por chamadas diretas à API:

- O ano letivo tem quatro bimestres, cada um com data e hora de abertura e encerramento definidas pelo administrador, sem sobreposição.
- O professor só registra ou edita acompanhamentos das turmas e disciplinas às quais está vinculado e somente dentro do período aberto do bimestre.
- Cada acompanhamento pertence a um único aluno, turma, disciplina, professor, bimestre e ano letivo, garantido por chave única no banco.
- A média respeita a escala de 0 a 10, com no máximo uma casa decimal, validada no servidor.
- Um acompanhamento pode ter várias tags, todas cadastradas pelo administrador.
- Somente registros publicados são visíveis ao responsável.
- Alterações administrativas são rastreáveis, com usuário, data e hora, estado anterior e posterior.
- Um responsável pode estar vinculado a vários alunos, e um aluno pode ter vários responsáveis.
- Um aluno só pode estar matriculado em uma turma por ano letivo.
- Cada disciplina tem um único professor por turma.
- Só se publica o que foi enviado; rascunhos não podem ser publicados diretamente.
- O professor só edita registros em rascunho ou devolvidos; depois de enviado, o registro fica com a administração.
- A ciência do responsável é única por aluno e bimestre.

3.4 Requisitos não funcionais

Os requisitos não funcionais do enunciado orientaram decisões técnicas em todo o projeto:

- **Responsividade:** interface adaptada a computador, tablet e celular, com menu lateral que vira barra inferior no celular.

- **Usabilidade e acessibilidade:** navegação consistente, mensagens claras de sucesso e erro, rótulos em todos os campos, uso por teclado, estados de carregamento e de lista vazia, e contraste alto no tema escuro.
- **Segurança:** senhas com hash bcrypt, validação de entradas, consultas parametrizadas contra injeção de SQL, escape automático do React contra XSS, token no cabeçalho em vez de cookie (o que evita CSRF), sessões com expiração e menor privilégio.
- **Privacidade e LGPD:** coleta mínima, acesso restrito por vínculo e proteção especial aos dados de crianças.
- **Integridade:** chaves estrangeiras, chaves únicas, restrições de domínio e transações.
- **Desempenho:** paginação, índices nas colunas usadas em filtros e junções, e consultas agregadas no banco.
- **Auditabilidade:** histórico de alterações e publicações.
- **Manutenibilidade:** camadas separadas, padrão de nomes em português, versionamento e README com instruções.
- **Compatibilidade:** testado nos navegadores atuais baseados em Chromium e em celular.
- **Disponibilidade e recuperação:** tratamento centralizado de erros, registros no console do servidor e banco em serviço gerenciado.

3.5 Backlog e critérios de aceitação

Os requisitos foram desdobrados em um backlog de 27 histórias de usuário, todas concluídas, com 18 regras de negócio numeradas e critérios de aceitação para cada história. Esse material está no documento "KFKA – Backlog, Requisitos, Regras de Negócio e Critérios de Aceitação", na pasta Documentos/Documentação do repositório.

Como exemplo, a história "Como professor, quero lançar a média e as tags de um aluno para registrar seu desenvolvimento no bimestre" tem, entre seus critérios de aceitação: o lançamento só é aceito dentro do período aberto; a média fora da escala é recusada com mensagem clara; o professor não consegue lançar para turmas que não são suas; e o registro é salvo como rascunho até ser enviado.

4. Tecnologias e ambiente de desenvolvimento

4.1 Escolha da tecnologia

O enunciado permitia escolher, no backend, entre Node.js, .NET ou Java; no banco, entre MySQL e PostgreSQL; e, no frontend, entre React, Angular e Vue. A equipe escolheu **Node.js com Express**, **MySQL** e **React**. A principal razão foi usar **JavaScript nas duas pontas** do sistema, o que reduz a quantidade de linguagens a dominar no semestre, facilita a troca de dados em JSON e permite que o mesmo integrante transite entre a API e a interface. O MySQL foi escolhido por ser o banco estudado na disciplina de Banco de Dados e por funcionar em versão portátil nas máquinas da faculdade.

4.2 Backend

- **Node.js 24** – ambiente de execução JavaScript no servidor.
- **Express 5** – framework para criar as rotas HTTP da API. Na versão 5, erros em funções assíncronas são tratados de forma mais simples.
- **mysql2** – driver de conexão com o MySQL, com pool de conexões, promessas e consultas parametrizadas.
- **bcrypt** – geração e verificação de hash de senhas.
- **jsonwebtoken** – emissão e validação dos tokens JWT.
- **dotenv** – leitura das variáveis de ambiente a partir do arquivo .env no ambiente local.
- **cors** – liberação controlada de acesso à API a partir do endereço do frontend em desenvolvimento.
- **PDFKit** – geração do relatório em PDF no servidor.
- **ExcelJS** – geração das planilhas .xlsx.
- **nodemon** – reinício automático do servidor durante o desenvolvimento.

4.3 Frontend

- **React 19** – biblioteca para construção da interface em componentes.
- **Vite** – ferramenta de desenvolvimento e empacotamento, com recarga instantânea.
- **React Router 7** – navegação entre telas sem recarregar a página.

- **lucide-react** – biblioteca de ícones em SVG.
- **CSS próprio** com variáveis (tokens de cor, raio e espaçamento) definidas a partir do guia de estilo do Figma, em vez de um framework pronto. A escolha deu controle total sobre o visual e manteve o pacote final leve.
- **ESLint** – verificação automática de padrões e erros comuns no código.

4.4 Banco de dados

Foi utilizado o **MySQL 8**. No laboratório da faculdade, onde não é possível instalar programas com permissão de administrador, a equipe usou a versão portátil em ZIP: o servidor era inicializado uma única vez com `mysqld --initialize-insecure` e ligado a cada sessão com `mysqld --console`. No notebook, foi usada a versão instalada (MySQL 8.4), que roda como serviço do Windows. Em produção, o banco é um serviço MySQL gerenciado pelo Railway.

4.5 Ambiente e ferramentas

O ambiente de desenvolvimento foi o Windows com PowerShell, Node.js 24.1, npm 11.3 e Git 2.55. Dois obstáculos do ambiente foram contornados logo no início: a política de execução do PowerShell bloqueava o comando `npm`, resolvido com o uso de `npm.cmd`; e a versão web do Postman não alcançava o `localhost`, resolvido com o Postman Desktop Agent e, depois, com o Postman desktop.

As portas padronizadas foram 3306 para o MySQL, 3001 para a API e 5173 para o frontend em desenvolvimento. Em produção, a API e a interface passam a responder no mesmo endereço.

5. Arquitetura do sistema

5.1 Visão geral

O sistema segue a arquitetura **cliente-servidor** com três partes: a interface React, executada no navegador do usuário; a API REST em Node.js, que concentra as regras de negócio e a segurança; e o banco de dados MySQL, que guarda os dados. A interface nunca acessa o banco diretamente: ela faz requisições HTTP à API, que valida o token, verifica as permissões, aplica as regras e só então consulta ou grava no banco.

Em produção, o mesmo servidor Node.js entrega a API e os arquivos já compilados da interface. Assim, o navegador acessa um único endereço, o que simplifica a implantação e dispensa configurações de CORS em produção.

```
Navegador (React)  --HTTP/JSON + token JWT-->  API Node.js/Express  --SQL-->  MySQL
                   <--------- JSON ---------                        <--------
```

5.2 Camadas do backend

O backend foi dividido em camadas, cada uma com uma responsabilidade única, o que facilita a manutenção e os testes:

- **Rotas (routes/):** recebem a requisição, leem e validam o formato dos dados, chamam o serviço ou repositório adequado e montam a resposta HTTP.
- **Middlewares (middlewares/):** funções executadas antes das rotas, como a verificação do token, a checagem do perfil e a validação de identificadores.
- **Serviços (services/):** concentram as regras de negócio que envolvem mais de uma tabela ou decisão, como o período do bimestre, o fluxo editorial, a recuperação de senha, a geração do PDF e a exportação.
- **Repositórios (repositories/):** isolam todo o SQL. Cada entidade tem um repositório com métodos como listar, buscar por id, criar, atualizar e inativar.
- **Modelos (models/):** classes que representam as entidades (Aluno, Professor, Turma, Acompanhamento...) e sabem se converter para JSON.
- **Núcleo (core/):** a classe `Database`, que encapsula o pool de conexões, e a classe principal `SistemaEscolar`.
- **Erros (errors/):** a classe `ErroDeNegocio`, que carrega o código HTTP e a mensagem de cada regra violada.

5.3 A classe principal SistemaEscolar

A classe `SistemaEscolar` é o ponto central do backend, desenvolvida como entrega da disciplina de Programação Orientada a Objetos. Ela orquestra a inicialização do sistema em uma ordem bem definida: conecta ao banco de dados, configura os middlewares globais (leitura de JSON, CORS), cria os repositórios e serviços injetando neles as dependências, registra as rotas de cada módulo, configura a entrega da interface compilada e, por fim, sobe o servidor na porta configurada.

Com isso, o arquivo `server.js` ficou reduzido a poucas linhas: ele cria uma instância de `SistemaEscolar`, informando a porta (a da nuvem, quando existir, ou a 3001 no computador), e chama o método de inicialização.

```
const SistemaEscolar = require('./core/SistemaEscolar');

// Na nuvem, a plataforma informa a porta pela variável PORT; no computador, usamos a
3001
const sistema = new SistemaEscolar(Number(process.env.PORT) || 3001);
sistema.iniciar();
```

5.4 Conceitos de orientação a objetos aplicados

- **Encapsulamento:** atributos privados com `#` (por exemplo, o pool de conexões dentro de `Database` e os repositórios dentro dos serviços), acessados apenas por métodos da própria classe.
- **Abstração:** os repositórios escondem o SQL; as rotas pedem "listar alunos" sem saber como a consulta é feita.
- **Injeção de dependência:** cada repositório recebe o objeto de banco no construtor, e cada serviço recebe os repositórios de que precisa. Isso deixa as classes independentes e mais fáceis de testar e trocar.
- **Responsabilidade única:** cada classe tem um papel claro (entidade, acesso a dados, regra de negócio ou orquestração).
- **Herança:** a classe `ErroDeNegocio` estende a classe nativa `Error`, acrescentando o código HTTP.
- **Métodos de instância e propriedades calculadas:** por exemplo, `Bimestre.situacao` calcula se o bimestre está aguardando, aberto ou encerrado, e `Usuario.ehAdministrador()` informa o perfil.

5.5 Estrutura de pastas

```
Projeto12/
  backend/
    core/          Database.js, SistemaEscolar.js
    database/      schema.sql
    errors/        ErroDeNegocio.js
    middlewares/   autenticacao.js, validacao.js
    models/        Usuario, Aluno, Professor, Responsavel, Turma, Disciplina, ...
    repositories/  um repositório por entidade + RelatorioRepository
    routes/        uma rota por módulo + auxiliares.js
    services/      Acompanhamento, Revisao, AreaResponsavel, Senha, GeradorPdf, ...
    scripts/       carregarDadosFicticios.js, criarAdmin.js
    server.js
  frontend/
    src/
      componentes/ Layout, Modal, Selo, EstadoTela, Paginacao, ...
      contexto/    AuthContext.jsx
      ganchos/     useCarregar.js
      paginas/     Home, Login, admin/, professor/, responsavel/
      servicos/    api.js
      utils/       formatar.js, media.js, baixarArquivo.js
  Documentos/      entregas por disciplina e documentação
  package.json     build e start usados na implantação
  README.md
```

5.6 Caminho de uma requisição

Para ilustrar a arquitetura, considere o professor salvando um lançamento. O navegador envia `PUT /professor/turmas/1/disciplinas/2/bimestres/4/alunos/7` com o token no cabeçalho e a descrição, a média e as tags no corpo. O middleware `verificarToken` confere o token e identifica o usuário; `somentePerfis('professor')` confirma o perfil; o validador de parâmetros confere que os identificadores são números inteiros positivos. A rota valida o corpo (descrição obrigatória, média de 0 a 10, tags como lista de números) e chama o `AcompanhamentoService`.

O serviço verifica se o usuário é um professor ativo, se leciona aquela disciplina naquela turma, se o bimestre existe e está aberto, se o aluno está matriculado na turma e se o registro ainda pode ser editado. Passadas as regras, grava o acompanhamento e as tags em uma transação, registra o histórico e devolve o registro atualizado. Se qualquer regra falhar, um `ErroDeNegocio` com o código adequado é convertido em uma resposta como `{ "erro": "O período de digitação deste bimestre está fechado." }`.

6. Banco de dados

6.1 Modelo conceitual

O modelo gira em torno de quatro grupos de entidades: as **pessoas** (usuários, alunos, professores e responsáveis), a **estrutura escolar** (áreas, disciplinas, turmas, tags e bimestres), os **vínculos** (aluno na turma, disciplina e professor na turma, responsável e aluno) e o **acompanhamento** (o registro bimestral, suas tags, o histórico e a ciência do responsável).

Os principais relacionamentos são: um usuário pode ser um professor ou um responsável (relação um para um); uma turma tem muitos alunos e um aluno tem uma turma por ano (muitos para muitos, com restrição por ano letivo); uma turma tem muitas disciplinas, cada uma com um professor; um responsável tem muitos alunos e um aluno tem muitos responsáveis; um acompanhamento liga aluno, turma, disciplina, professor e bimestre, e tem muitas tags.

6.2 Tabelas

O script `backend/database/schema.sql` cria dezessete tabelas:

- **usuarios** – contas de acesso: nome, e-mail único, hash da senha, perfil (administrador, professor ou responsável), situação ativo/inativo e data de criação.
- **alunos** – nome, data de nascimento, matrícula única e situação.
- **professores** – registro funcional único, ligado um a um a um usuário.
- **responsaveis** – telefone opcional, ligado um a um a um usuário.
- **responsavel_aluno** – vínculo entre responsável e aluno, com o parentesco.
- **areas_disciplina** – áreas do conhecimento (por exemplo, Linguagens).
- **disciplinas** – nome e área a que pertence.
- **turmas** – ano/série (1º ao 9º), identificação (A, B...), turno (manhã, tarde ou integral) e ano letivo.
- **tags** – nome e descrição das tags usadas nos acompanhamentos.
- **turma_aluno** – matrícula do aluno na turma, com o ano letivo para impedir duas turmas no mesmo ano.

- **turma_disciplina_professor** – disciplina alocada na turma e professor responsável.
- **bimestres** – número (1 a 4), ano letivo, abertura e encerramento da digitação.
- **acompanhamentos** – o registro bimestral: aluno, turma, disciplina, professor, bimestre, descrição, média, status (rascunho, enviado, devolvido ou publicado), observação da revisão, revisor e datas de envio, revisão e publicação.
- **acompanhamento_tag** – tags marcadas em cada acompanhamento.
- **historico_auditoria** – usuário, ação, entidade, estado anterior e posterior (em JSON) e data e hora.
- **ciencias** – ciência e observação do responsável por aluno e bimestre.
- **recuperacoes_senha** – pedidos de recuperação, com o hash do token, a validade e o uso.

6.3 Integridade dos dados

A integridade foi garantida no próprio banco, e não só no código:

- **Chaves estrangeiras** em todos os relacionamentos, impedindo registros órfãos (por exemplo, um acompanhamento de um aluno inexistente).
- **Chaves únicas** para impedir duplicidade: e-mail do usuário, matrícula do aluno, registro do professor, aluno por turma, aluno por ano letivo, disciplina por turma, bimestre por número e ano, acompanhamento por aluno, turma, disciplina e bimestre, e ciência por aluno e bimestre.
- **Tipos de domínio** com `ENUM` para perfil, turno e status, e `CHECK` para a faixa da média, o número do bimestre, o ano/série da turma e a ordem entre abertura e encerramento do bimestre.
- **Transações** nas operações que alteram várias tabelas, como criar um professor junto com sua conta de usuário, ou salvar um acompanhamento com suas tags e o registro de histórico. Se uma parte falha, tudo é desfeito.
- **Exclusão lógica** (campo `ativo`) nos cadastros, preservando o histórico dos acompanhamentos já registrados.

6.4 Índices

Além dos índices criados automaticamente pelas chaves primárias, estrangeiras e únicas, foram criados índices específicos para as consultas mais

frequentes, justificados no documento de índices entregue à disciplina de Banco de Dados:

- `idx_acomp_status_bimestre (status, bimestre_id)` – fila de revisão e relatórios de situação por bimestre.
- `idx_acomp_professor (professor_id, turma_id, disciplina_id)` – painel de lançamentos do professor.
- `idx_acomp_aluno_status (aluno_id, status)` – relatórios publicados do aluno na área do responsável.
- `idx_tdp_professor (professor_id)` – turmas e disciplinas de cada professor.
- `idx_acg_tag (tag_id)` – relatório de frequência de tags.
- `idx_hist_entidade (entidade, entidade_id, criado_em)` – linha do tempo do histórico de um registro.
- `idx_rec_usuario (usuario_id)` – pedidos de recuperação de senha de um usuário.

6.5 Auditoria

A tabela `historico_auditoria` guarda cada ação relevante com o usuário que a executou, a data e a hora, a entidade e o identificador afetados, o estado anterior e o estado posterior em formato JSON. Isso atende à exigência do enunciado de que alterações administrativas sejam rastreáveis e permite reconstruir, na tela de revisão, toda a vida de um acompanhamento: quem criou, quando foi enviado, o que a administração alterou, se foi devolvido e por quê, e quando foi publicado.

6.6 Carga de dados fictícios

O script `backend/scripts/carregarDadosFicticios.js`, executado com `npm run dados`, recria o banco do zero e o popula com um cenário realista para demonstração: 24 alunos, 3 turmas, 5 professores, 12 famílias com responsáveis vinculados, áreas, disciplinas, tags, os quatro bimestres de 2026 (os três primeiros encerrados e publicados, o quarto aberto para digitação), acompanhamentos em todos os estados do fluxo, histórico de auditoria e ciências registradas. O script exige o parâmetro `--confirmar`, para evitar que o banco seja apagado por engano. O mesmo script foi usado para carregar o banco de produção.

6.7 Fuso horário

Como os prazos dos bimestres são definidos em data e hora, o fuso horário é importante. Cada conexão do pool executa `SET time_zone = '-03:00'`, e o servidor em produção roda com `TZ=America/Sao_Paulo`. Assim, um bimestre que encerra às 23h59 encerra às 23h59 de Brasília, independentemente de onde o servidor está hospedado.

7. Construção do backend (API)

7.1 Padrões gerais da API

A API segue o estilo REST, com recursos identificados por endereços (`/alunos`, `/turmas/5/alunos`) e operações pelos métodos HTTP: GET para consultar, POST para criar ou executar uma ação, PUT para alterar e DELETE para inativar ou desvincular. Todas as respostas são em JSON.

As listas grandes (alunos, professores e responsáveis) são **paginadas**: aceitam `?pagina=` e `?porPagina=` (padrão de 20 itens, máximo de 100) e devolvem `{ itens, pagina, porPagina, total, totalPaginas }`. Também aceitam `?busca=` para filtrar por nome, e-mail ou matrícula. As listas pequenas (áreas, disciplinas, turmas e tags) são devolvidas inteiras.

Os erros seguem um **formato único**, `{ "erro": "mensagem" }`, com mensagens em português pensadas para o usuário final, e os códigos HTTP indicam a natureza do problema: 400 para dado inválido, 401 para falta de autenticação, 403 para falta de permissão ou prazo encerrado, 404 para registro não encontrado e 409 para conflito com dados existentes ou com o fluxo. Erros inesperados viram 500 com mensagem genérica, sem expor detalhes internos ao usuário.

7.2 Autenticação e autorização

O login (`POST /auth/login`) busca o usuário pelo e-mail, verifica se está ativo e compara a senha digitada com o hash guardado usando `bcrypt.compare`. Se tudo estiver correto, emite um token JWT assinado com a chave secreta do servidor, contendo o identificador e o perfil do usuário, com validade de 8 horas. Se o e-mail não existir ou a senha estiver errada, a resposta é a mesma ("E-mail ou senha inválidos."), para não revelar quais e-mails estão cadastrados.

Dois middlewares protegem as rotas. O `verificarToken` lê o cabeçalho `Authorization: Bearer <token>`, valida a assinatura e a validade e anexa o usuário à requisição; sem token, a resposta é 401 "Token não fornecido."; com token inválido ou vencido, 401 "Token inválido ou expirado.". O `somentePerfis(...)` é uma fábrica de middlewares que recebe os perfis permitidos e responde 403 para os demais. Um terceiro middleware, `validarId`, aplicado com `router.param`, garante que todo identificador na URL seja um número inteiro positivo, respondendo 400 caso contrário.

Além do perfil, o sistema verifica o **vínculo**: o professor só acessa turmas e disciplinas que leciona, e o responsável só acessa alunos vinculados a ele. Essa checagem acontece nos serviços, consultando o banco a cada requisição.

7.3 Recuperação e troca de senha

Em `POST /auth/recuperar-senha`, o usuário informa o e-mail. Se ele existir, o sistema gera um token aleatório, guarda apenas o **hash** desse token no banco, com validade de 30 minutos, e monta o link de redefinição. A resposta é sempre a mesma, exista o e-mail ou não. Em `POST /auth/redefinir-senha`, o usuário envia o token do link e a nova senha; o sistema confere o hash, a validade e se o link já foi usado, aplica a regra de força (8 a 72 caracteres, com letra e número) e grava o novo hash. O usuário logado também pode trocar a senha em `POST /auth/alterar-senha`, informando a senha atual.

O envio do e-mail foi isolado na classe `NotificadorEmail`. No projeto, o envio é simulado: o conteúdo do e-mail, com o link, é exibido no console do servidor. Para enviar de verdade, basta trocar a implementação dessa classe por uma biblioteca de e-mail e um servidor SMTP, sem alterar o restante do código.

7.4 Cadastros

Os sete cadastros seguem o mesmo padrão de rotas: GET para listar, POST para criar, PUT `/:id` para alterar e DELETE `/:id` para inativar. As validações incluem campos obrigatórios, tamanhos máximos, formato de e-mail, normalização do telefone (apenas dígitos), datas válidas e unicidade (matrícula, registro funcional, e-mail). Ao cadastrar um professor ou responsável, o sistema cria também a conta de usuário com a senha inicial em hash, dentro de uma transação.

7.5 Vínculos

As rotas `/turmas/:id/alunos`, `/turmas/:id/disciplinas` e `/responsaveis/:id/alunos` permitem listar, criar e remover vínculos. A troca do professor de uma disciplina na turma é feita com PUT. As restrições de unicidade do banco, somadas a verificações nos repositórios, impedem matrícula duplicada, aluno em duas turmas no mesmo ano, disciplina duplicada na turma e vínculo repetido entre responsável e aluno, sempre com mensagens claras (código 409).

7.6 Bimestres

`GET /bimestres?anoLetivo=2026` lista os quatro bimestres com a situação calculada (aguardando, aberto ou encerrado). `PUT /bimestres/:ano/:numero` cria ou altera o período de um bimestre, validando que o encerramento seja posterior à abertura e que não haja sobreposição com os demais bimestres do ano. A consulta é liberada também para professores e responsáveis, que precisam saber o bimestre em andamento.

7.7 Lançamentos do professor

O professor consulta suas turmas e disciplinas em `GET /professor/turmas`. Ao escolher uma combinação de turma, disciplina e bimestre, `GET /professor/turmas/:t/disciplinas/:d/bimestres/:n` devolve o painel com a situação do período e a lista de alunos, cada um com o estado do seu acompanhamento. O lançamento individual é lido e gravado em `/alunos/:alunoId` (GET e PUT). O envio para revisão é feito em `POST /professor/acompanhamentos/:id/enviar`, que também confere se o período está aberto. Os retornos das famílias aparecem em `GET /professor/retornos`.

7.8 Fluxo editorial e revisão

O fluxo editorial foi modelado como uma **máquina de estados** com quatro estados:

- **rascunho** – criado e editável pelo professor;
- **enviado** – aguardando a revisão da administração; o professor não edita mais;
- **devolvido** – a administração pediu ajustes, com uma observação; o professor edita e reenvia;
- **publicado** – visível ao responsável.

As transições permitidas são: rascunho para enviado (professor), devolvido para enviado (professor), enviado para devolvido (administrador) e enviado para publicado (administrador). Qualquer outra transição é recusada com código 409, por exemplo tentar publicar um rascunho. A administração pode também alterar o conteúdo (descrição, média e tags) durante a revisão, e cada alteração fica registrada no histórico com o estado anterior e o posterior.

As rotas de revisão são `GET /revisao/acompanhamentos` (fila com filtros e paginação), `GET /revisao/acompanhamentos/:id` (detalhe com o histórico completo), `PUT /revisao/acompanhamentos/:id` (alteração administrativa), `POST /:id/devolver` (com observação obrigatória) e `POST /:id/publicar`.

7.9 Área do responsável

`GET /responsavel/alunos` lista os alunos vinculados ao responsável logado, com turma e parentesco. `GET /responsavel/alunos/:id/relatorio?anoLetivo=2026&bimestre=n` devolve o relatório publicado do bimestre, com cada disciplina, média, tags e descrição, além da situação da ciência. `GET /responsavel/alunos/:id/relatorio/pdf` gera o PDF. `POST /responsavel/alunos/:id/ciencia` registra a ciência e a observação opcional.

7.10 Geração de PDF

O serviço `GeradorPdf` usa a biblioteca PDFKit para desenhar o relatório diretamente no servidor e enviá-lo como arquivo para download. O documento traz o cabeçalho com o nome do sistema, a identificação do aluno, da turma, do ano letivo e do bimestre, e uma seção por disciplina com a média, as tags e a descrição publicada. Por ser gerado no servidor, o PDF é idêntico em qualquer navegador ou aparelho, e só pode ser gerado para alunos vinculados e relatórios publicados.

7.11 Relatórios gerenciais e exportação

O `RelatorioRepository` concentra consultas agregadas executadas no banco (com `GROUP BY`, `AVG` e `COUNT`), o que é mais eficiente que trazer todos os registros para o servidor. As rotas são `/relatorios/medias` (média por turma e disciplina), `/relatorios/status` (quantidade por estado do fluxo), `/relatorios/tags` (frequência de cada tag) e `/relatorios/retornos` (ciências e observações das famílias). A exportação, em `/relatorios/exportar?formato=xlsx` ou `csv`, é feita pelo serviço `ExportadorPlanilha`, com a biblioteca ExcelJS para .xlsx e geração própria para .csv, com separador ponto e vírgula e marca de codificação UTF-8, o que faz o Excel em português abrir o arquivo com acentos e colunas corretas.

8. Construção do frontend

8.1 Organização

O frontend é uma aplicação de página única (SPA) em React. As telas ficam em `paginas/`, separadas por módulo (admin, professor e responsável); os componentes reutilizáveis em `componentes/`; a comunicação com a API em `servicos/api.js`; o estado de login em `contexto/AuthContext.jsx`; e funções utilitárias de formatação em `utils/`.

8.2 Comunicação com a API

A função `api()` centraliza todas as chamadas: monta o endereço, acrescenta o token no cabeçalho, converte o corpo para JSON, interpreta a resposta e, em caso de erro, lança uma exceção com a mensagem enviada pelo servidor, que as telas exibem diretamente ao usuário. Se o token vencer, o usuário é levado de volta ao login. O endereço da API vem da variável `VITE_API_URL`; em produção, como API e interface estão no mesmo servidor, usa-se o endereço relativo.

Downloads (PDF, .xlsx e .csv) passam pela função `baixarArquivo`, que faz a requisição autenticada, recebe o arquivo como blob e dispara o download no navegador com o nome correto.

8.3 Autenticação e rotas protegidas

O `AuthContext` guarda o token e os dados do usuário logado, disponibiliza as funções de entrar e sair e restaura a sessão ao recarregar a página. O componente `RotaProtegida` envolve as rotas de cada módulo: se não há usuário, redireciona ao login; se o perfil não corresponde, redireciona à área correta. Assim, um professor que digitar /admin no navegador é levado para a sua própria área. Essa proteção é apenas de navegação; a segurança real está na API, que valida cada requisição.

8.4 Hook de carregamento

Para padronizar o carregamento de dados, foi criado o hook `useCarregar`, que executa uma função assíncrona e devolve `{ dados, carregando, atualizando, erro, recarregar }`. Com ele, toda tela trata da mesma forma os estados de carregamento, erro (com botão "Tentar novamente") e atualização (a lista atual fica levemente

apagada enquanto os novos dados chegam, sem piscar a tela). O componente `EstadoTela` exibe esses estados de forma consistente.

8.5 Componentes reutilizáveis

- **Layout** – estrutura das áreas logadas, com menu lateral no computador e barra inferior no celular, e cabeçalho com o nome do usuário.
- **Modal** – janela sobreposta para formulários e confirmações, que fecha com a tecla Esc ou pelo botão de fechar.
- **Selo** – etiqueta colorida para os estados (rascunho, enviado, devolvido, publicado, aberto, encerrado...).
- **BarraProgresso** – mostra quanto do período de digitação já passou.
- **CartaoNumero** – cartão com um indicador numérico.
- **Paginacao** – navegação entre páginas das listas.
- **CartaoAcesso** – moldura das telas de login e recuperação de senha.
- **EstadoTela** – carregando, erro e vazio.

8.6 Telas do módulo Administrador

- **Início:** resumo do ano letivo, com a quantidade de acompanhamentos aguardando revisão, devolvidos e em rascunho, o bimestre aberto com prazo e barra de progresso, totais de alunos, turmas e professores, atalhos e a situação dos quatro bimestres.
- **Cadastros:** uma única tela com sete abas (alunos, professores, responsáveis, áreas, disciplinas, turmas e tags), construída a partir de um arquivo de configuração. Em vez de escrever sete telas quase iguais, cada aba é descrita (endereço da API, colunas e campos do formulário) e um único componente a monta. Inclui busca com espera de 300 milissegundos após a digitação, paginação, formulário em janela e confirmação de inativação.
- **Turma:** abas de alunos matriculados e de disciplinas com professor, com inclusão, remoção e troca do professor direto na tabela.
- **Responsável:** vínculo de alunos com o parentesco, com sugestões (mãe, pai, avó...) que aceitam texto livre.
- **Bimestres:** edição da abertura e do encerramento de cada bimestre, com a situação atual.

- **Revisão:** fila de acompanhamentos com filtros por estado, turma e bimestre; tela de revisão com o conteúdo editável, a linha do tempo do histórico e as ações de salvar alteração, devolver (com observação obrigatória) e publicar.
- **Relatórios:** médias, situação do fluxo, frequência das tags e retornos das famílias, com exportação para .xlsx e .csv.

8.7 Telas do módulo Professor

- **Início:** bimestre aberto com prazo, pendências (rascunhos e devolvidos) e retornos recentes das famílias.
- **Minhas turmas:** lista das turmas e disciplinas que o professor leciona.
- **Lançamentos:** para a turma, disciplina e bimestre escolhidos, a lista de alunos com o estado de cada registro e a situação do período.
- **Formulário de lançamento:** descrição com contador de caracteres, média validada e seleção de várias tags, com os botões salvar rascunho e enviar para revisão. Quando o registro foi devolvido, a observação da administração aparece em destaque; quando o período está fechado ou o registro já foi enviado, o formulário fica somente para leitura.

8.8 Telas do módulo Pai/Responsável

- **Início:** cartões com os filhos vinculados e a situação do último relatório publicado.
- **Relatório do aluno:** escolha do bimestre, cartões por disciplina com média, tags e descrição, registro de ciência com observação opcional e botão para baixar o PDF.

8.9 Telas abertas e página inicial

As telas sem login são a **página inicial pública**, que apresenta o sistema, explica o fluxo em quatro passos, os três perfis e os cuidados de segurança, com chamadas para o acesso; o **login**; o **esqueci minha senha**, que confirma o envio sem revelar se o e-mail existe; e a **redefinição de senha**, que mostra as regras da senha e trata o caso de link inválido ou expirado.

8.10 Design e guia de estilo

O visual segue o guia de estilo definido no Figma: tema escuro com fundo quase preto, superfícies em tons de cinza azulado, ciano como cor de destaque e

cores semânticas para sucesso (verde), alerta (amarelo) e perigo (vermelho). As cores, raios de borda e medidas foram transformados em variáveis CSS (tokens), como `--fundo`, `--superficie`, `--ciano` e `--raio`, usadas em todo o código. Isso garante consistência e permite ajustar o tema em um único lugar. A tipografia é sem serifa, com hierarquia clara entre títulos, subtítulos e textos de apoio.

Inicialmente, a equipe pretendia converter as telas do Figma diretamente em código. Durante o projeto, optou-se por construir as telas em React a partir dos desenhos e do guia de estilo, e depois atualizar os protótipos no Figma para refletir o sistema final, o que deixou os protótipos e o produto alinhados.

8.11 Responsividade e acessibilidade

O layout usa grades flexíveis e pontos de quebra para tablet e celular. No celular, o menu lateral vira uma barra fixa inferior, as grades de cartões passam a uma coluna e as tabelas escondem colunas secundárias, mantendo o essencial legível. Para acessibilidade, todos os campos têm rótulo, os botões de ícone têm descrição (`aria-label`), as mensagens de erro usam `role="alert"`, as abas usam os papéis `tablist` e `tab`, as janelas modais fecham com Esc, e o contraste do texto foi escolhido para leitura confortável no tema escuro.

9. Segurança e privacidade (LGPD)

9.1 Medidas de segurança

- **Senhas:** guardadas apenas como hash bcrypt com custo 10; nunca em texto puro, nem em registros do sistema. Regra mínima de força: 8 a 72 caracteres, com letra e número.
- **Sessão:** token JWT assinado, com validade de 8 horas, enviado no cabeçalho Authorization.
- **Menor privilégio:** cada rota declara os perfis permitidos, e os serviços checam o vínculo do professor e do responsável.
- **Sem autocadastro:** só o administrador cria contas, impedindo que estranhos acessem dados de alunos.
- **Mensagens que não revelam dados:** o login e a recuperação de senha respondem igual para e-mails existentes e inexistentes; o acesso a aluno não vinculado responde "não encontrado".
- **Injeção de SQL:** todas as consultas são parametrizadas. Uma busca com `' OR 1=1 --` devolve lista vazia.
- **XSS:** o React escapa automaticamente todo texto exibido, e o sistema não usa inserção de HTML bruto.
- **CSRF:** como o token vai no cabeçalho, e não em cookie, o navegador não o envia automaticamente em requisições forjadas por outros sites.
- **Validação no servidor:** toda entrada é validada na API, mesmo que a interface já valide.
- **Recuperação de senha:** token aleatório, guardado como hash, com validade de 30 minutos e uso único.
- **Segredos fora do código:** senhas do banco e chave do JWT ficam em variáveis de ambiente; o arquivo .env está no .gitignore e nunca foi enviado ao GitHub; o repositório tem apenas o modelo .env.example.
- **Banco protegido em produção:** o acesso público ao MySQL do Railway foi aberto apenas para a carga dos dados fictícios e removido em seguida; o banco só é alcançado pela rede interna da plataforma.
- **Dependências:** foi executado `npm audit fix` para corrigir vulnerabilidades conhecidas nas bibliotecas.

9.2 Privacidade e LGPD

Por tratar dados de crianças e adolescentes, o sistema adota a **coleta mínima**: do aluno, apenas nome, data de nascimento e matrícula; do responsável, nome, e-mail e telefone opcional. O acesso é restrito por vínculo, de modo que cada família vê apenas os próprios filhos e cada professor apenas as próprias turmas. Somente informações revisadas e publicadas pela escola chegam às famílias. A inativação é lógica para preservar o histórico, e o histórico de auditoria documenta quem alterou o quê. Em produção, foram usados exclusivamente **dados fictícios**.

Para uso real em uma escola, seriam necessários ainda: termo de consentimento dos responsáveis, política de privacidade publicada, definição de prazos de retenção e procedimento para atender pedidos de acesso e exclusão dos titulares.

10. Testes e validação

10.1 Estratégia

Os testes acompanharam todo o desenvolvimento. Cada bloco implementado era testado antes do commit, e ao final foi executada uma bateria completa, registrada no documento "KFKA – Plano e Evidências de Testes", na pasta Documentos/Documentação do repositório. Foram combinados quatro tipos de teste:

- **Testes manuais da API no Postman:** cada rota foi chamada com dados válidos e inválidos, com e sem token e com tokens de perfis diferentes, conferindo o código HTTP e a mensagem.
- **Testes manuais da interface:** cada tela foi usada nos três perfis, no computador e no celular, conferindo fluxos, mensagens e responsividade.
- **Testes ponta a ponta automatizados:** durante o desenvolvimento, roteiros com a ferramenta Playwright executaram no navegador o cenário completo do sistema (login nos três perfis, lançamento, envio, revisão, devolução, publicação, consulta, ciência, PDF e exportação), tanto em modo de desenvolvimento quanto em modo de produção.
- **Testes no ambiente publicado:** após a implantação, o cenário completo foi repetido no endereço público, incluindo recarregar telas internas, baixar o PDF e usar o celular.

10.2 Casos de teste

O plano reúne 64 casos de teste, organizados por requisito e por perfil. Os grupos principais são:

- **Autenticação:** login válido e inválido, mensagem genérica, sessão expirada, recuperação de senha com link válido, expirado e já usado, senha fraca.
- **Controle de acesso:** cada perfil tentando acessar rotas dos outros; professor em turma que não é sua; responsável em aluno não vinculado; requisições sem token e com token inválido.
- **Cadastros e vínculos:** criação, edição, busca, paginação e inativação; matrícula, e-mail e registro duplicados; aluno em duas turmas; disciplina duplicada.
- **Bimestres:** período válido, encerramento antes da abertura, sobreposição, situação calculada.

- **Acompanhamento:** média fora da escala, com duas casas decimais e não numérica; descrição vazia e longa demais; tags inexistentes; lançamento com período fechado.
- **Fluxo editorial:** todas as transições válidas e as inválidas (publicar rascunho, editar enviado, devolver sem observação), e o registro no histórico.
- **Responsável:** visualização apenas de publicados, PDF, ciência repetida e observação visível ao professor.
- **Relatórios e exportação:** valores agregados conferidos com o banco e abertura do .xlsx e do .csv no Excel.
- **Segurança:** injeção de SQL na busca, identificadores inválidos e texto com marcação HTML exibido como texto.
- **Interface:** responsividade em celular, tablet e computador; navegação por teclado; mensagens de erro e de sucesso.

10.3 Exemplos de resultados

Alguns resultados reais registrados nos testes da API:

- Requisição sem token: 401, "Token não fornecido.".
- Professor acessando a lista de alunos: 403, "Você não tem permissão para acessar este recurso.".
- Responsável pedindo aluno de outra família: 404, "Aluno não encontrado entre os seus vinculados.".
- Identificador "abc" na URL: 400, "Identificador inválido. Use um número inteiro positivo.".
- Média 11: 400, "A média deve ser um número de 0 a 10, com no máximo uma casa decimal.".
- Lançamento com bimestre fechado: 403, "O período de digitação deste bimestre está fechado.".
- Matrícula repetida: 409, "Já existe um aluno com essa matrícula.".
- Bimestre sobreposto: 409, "O período se sobrepõe ao 4º bimestre de 2026.".
- Ciência repetida: 409, "A ciência deste bimestre já foi registrada.".
- Busca com `' OR 1=1 --`: 200, com lista vazia.

10.4 Defeitos encontrados e corrigidos

Os testes encontraram e permitiram corrigir dez defeitos antes da entrega. Entre eles: rotas de consulta de tags e bimestres que bloqueavam professores e responsáveis, que precisavam desses dados; cadastro de professor e responsável que aceitava e-mail mal formatado e senha fraca; rotas de alteração duplicadas em dois arquivos; o retorno de uma família que não aparecia para o professor; e erros de nome de arquivo e de variável removida por engano no frontend. Todos foram corrigidos e testados novamente.

10.5 Critério de sistema funcional

O enunciado define que o sistema é funcional quando o seguinte cenário pode ser executado de ponta a ponta. Ele foi executado com sucesso no ambiente local e no ambiente publicado:

1. O administrador cadastra a estrutura escolar e abre o bimestre.
2. O professor registra e envia o acompanhamento.
3. O administrador revisa e publica.
4. O responsável consulta o relatório e gera o PDF.
5. O administrador extrai os dados para o Excel.

11. Implantação em nuvem

11.1 Plataforma

O sistema foi publicado na plataforma **Railway**, que oferece hospedagem de aplicações Node.js e bancos MySQL gerenciados, com variáveis de ambiente e endereço público com HTTPS. O endereço do sistema é https://projeto12-production.up.railway.app.

11.2 Arquitetura da implantação

O projeto no Railway tem dois serviços. O serviço **MySQL** guarda o banco de dados. O serviço **projeto12** executa o servidor Node.js, que responde às rotas da API e também entrega a interface React já compilada. Para isso, a classe `SistemaEscolar` ganhou o método `#servirFrontend()`: ele publica os arquivos da pasta `frontend/dist` e, para qualquer requisição de página (que aceita HTML) que não seja da API, devolve o `index.html`, deixando o React Router cuidar da navegação. Assim, recarregar uma tela interna, como /admin/revisao, funciona normalmente.

11.3 Preparação do código

- A porta passou a vir de `process.env.PORT`, informada pela plataforma.
- O fuso horário foi fixado em America/Sao_Paulo no servidor e em -03:00 em cada conexão com o banco.
- O frontend passou a usar endereço relativo para a API em produção.
- Foi criado um `package.json` na raiz do repositório, com o comando de build (instala e compila o frontend e instala as dependências do backend) e o comando de start (sobe o backend).
- Foi criado o arquivo `backend/.env.example` com as variáveis necessárias.

11.4 Variáveis de ambiente

No serviço projeto12 foram configuradas as variáveis `DB_HOST`, `DB_PORT`, `DB_USER` e `DB_PASSWORD` como referências às variáveis do serviço MySQL (sem digitar a senha), `DB_NAME=sistema_escolar`, `JWT_SECRET` com um valor longo e aleatório diferente do usado localmente, `TZ=America/Sao_Paulo` e `FRONTEND_URL` com o endereço público.

11.5 Passo a passo realizado

1. Criação da conta e do projeto no Railway e adição do serviço MySQL.

2. Ativação temporária do acesso público ao MySQL para executar, a partir do notebook, o script de criação das tabelas e a carga de dados fictícios.
3. Remoção do acesso público ao MySQL logo após a carga.
4. Tentativa de conectar o repositório do GitHub ao Railway, que não foi possível porque a organização da turma exige permissão de administrador para instalar o aplicativo do Railway.
5. Instalação da linha de comando do Railway e publicação a partir do notebook com `railway.cmd up --service projeto12`, que envia o código, executa o build e sobe o servidor.
6. Configuração das variáveis de ambiente e geração do domínio público.
7. Testes completos no endereço público com os três perfis.

Para atualizar o sistema publicado, basta atualizar o código local com `git pull` e repetir `railway.cmd up --service projeto12` a partir da pasta do projeto.

12. Relação com as disciplinas do semestre

12.1 Desenvolvimento Web Full Stack

A disciplina está presente em todo o sistema: API REST em Node.js e Express, interface React, integração entre as duas camadas, autenticação com JWT, rotas protegidas, validação e erros padronizados, segredos em variáveis de ambiente e publicação em nuvem. Na segunda entrega, a equipe documentou com evidências os sete itens pedidos: rotas principais, persistência, autenticação com rota protegida, validação, segredos fora do código, aplicação publicada e commits de todos os integrantes.

12.2 Banco de Dados

A modelagem das dezessete tabelas, as chaves, as restrições, os índices, as transações, as consultas agregadas dos relatórios e o script de carga aplicam o conteúdo da disciplina. Foi entregue um documento de índices, com onze índices justificados pelos requisitos funcionais, relacionando cada índice à consulta que ele acelera.

12.3 Programação Orientada a Objetos

O backend inteiro foi escrito em classes, com encapsulamento, injeção de dependência, herança e responsabilidade única. A entrega da disciplina foi a classe principal `SistemaEscolar`, que integra e orquestra as demais partes do sistema, acompanhada da explicação dos conceitos aplicados.

12.4 Estrutura de Dados

A disciplina pediu uma aplicação em C# que simulasse a paginação de alunos usando listas populadas a partir de um banco relacional. Essa entrega foi desenvolvida por um integrante do grupo e está no repositório. No sistema web, os conceitos aparecem na paginação das listas (cálculo de deslocamento e total de páginas), no uso de conjuntos (`Set`) para filtrar rapidamente os alunos e disciplinas ainda não vinculados, em mapas e listas para montar relatórios agrupados por disciplina e bimestre, e na máquina de estados do fluxo editorial.

12.5 Design de Interface Digital

O grupo produziu no Figma os protótipos das telas, o guia de estilo e o design system, que serviram de base para a interface. As cores, tipografia, componentes (botões, campos, selos, cartões, abas e janelas) e estados (carregando, vazio, erro e sucesso) foram implementados no código como tokens e componentes reutilizáveis. A página inicial pública foi desenhada pelo grupo e incorporada ao sistema na versão final.

13. Dificuldades enfrentadas e soluções

O projeto passou por vários obstáculos técnicos e de organização. Registrá-los é importante, porque cada um gerou uma prática que a equipe passou a seguir.

13.1 Ambiente restrito no laboratório

As máquinas da faculdade não permitem instalar programas nem usar permissão de administrador. A solução foi usar o MySQL em versão portátil (ZIP), inicializado manualmente, e o `npm.cmd` para contornar a política de execução do PowerShell. A conexão do Workbench às vezes sumia entre as sessões e precisava ser recriada.

13.2 Troca de máquina com entrega urgente

Em uma das entregas, foi preciso trocar de computador e recriar tudo do zero: clonar o repositório, instalar dependências, recriar o .env, instalar um novo MySQL, executar o script do banco e recriar os arquivos que ainda não tinham sido enviados ao GitHub. A lição foi fazer commit logo após cada trecho que funciona, prática seguida até o fim do projeto.

13.3 Reorganização acidental do repositório

Um commit de entrega moveu toda a pasta do backend para dentro da pasta de documentos da disciplina de POO, deixando o backend da raiz incompleto. Os arquivos foram recuperados e copiados de volta, e foram corrigidos o nome de um arquivo (`Database.js`, cuja letra maiúscula havia sido trocada) e uma rota que voltara à versão antiga. A partir daí, a equipe combinou que a pasta do backend na raiz é a fonte oficial do código, e as pastas de entrega guardam apenas documentos.

13.4 Erros recorrentes de iniciante

Ao longo do caminho, apareceram erros comuns que viraram aprendizado: arquivo editado mas não salvo no VS Code; pasta criada fora do repositório; `require` dentro de um método; tabela inexistente porque faltou o `USE sistema_escolar`; o operador `<` que não funciona no PowerShell (substituído por `Get-Content` com pipe); o comando executado na pasta errada; arquivos criados com nome errado (letra minúscula ou vírgula no lugar do ponto); trecho sugerido automaticamente pelo editor

chamando um método que não existia; e o editor Vim que abre durante um `git pull` com merge (saída com `:wq`).

13.5 Erro 500 no cadastro de professores

O cadastro de professores, que cria também a conta de usuário, retornou erro 500 por um período. O diagnóstico, feito lendo a mensagem completa no console do servidor, levou à correção, e a operação passou a usar uma transação, garantindo que o usuário e o professor sejam criados juntos ou nenhum dos dois.

13.6 Falha de segurança nas rotas de usuários

Nas primeiras versões, as rotas de cadastro e listagem de usuários eram públicas, o que permitiria a qualquer pessoa criar uma conta ou ver a lista de usuários. A falha foi identificada e corrigida antes de qualquer demonstração: as rotas passaram a exigir token de administrador, e o cadastro de contas ficou restrito aos cadastros de professores e responsáveis.

13.7 Implantação

A implantação trouxe desafios próprios: a organização da turma no GitHub bloqueou a integração automática com o Railway, contornada com a linha de comando; o script do Railway também foi bloqueado pelo PowerShell, contornado com `railway.cmd`; o vínculo da linha de comando ficou apontando para o serviço do banco, exigindo informar sempre `--service projeto12`; e as variáveis do banco apareciam como referências, sendo necessário consultar cada uma individualmente para a carga de dados. Todos os passos foram feitos sem expor senhas em prints ou mensagens.

13.8 Prazo

O projeto chegou à reta final com a maior parte do sistema por construir. A equipe reorganizou o trabalho em blocos diários, priorizando primeiro o que o critério de "sistema funcional" exige, depois o frontend completo e, por fim, testes, implantação e documentação. O sistema foi concluído, publicado e documentado dentro do prazo.

14. Resultados

O resultado é um sistema completo, funcional e publicado, que atende aos doze requisitos funcionais e às regras de negócio do enunciado. Em números:

- 3 perfis de usuário, cada um com sua área e permissões;
- 66 rotas na API, organizadas em 14 módulos;
- 17 tabelas no banco, com chaves, restrições e 7 índices específicos;
- 10 classes de modelo, 12 repositórios e 7 serviços no backend;
- 18 telas no frontend, incluindo a página inicial pública e as de login e recuperação de senha;
- cerca de 8.600 linhas de código-fonte próprias (backend, frontend, CSS e SQL);
- 64 casos de teste documentados e 10 defeitos corrigidos;
- mais de 100 commits no repositório, com contribuições de todos os integrantes;
- sistema publicado em nuvem com dados fictícios de 24 alunos, 3 turmas, 5 professores e 12 famílias.

Os entregáveis do projeto e sua situação são: repositório versionado com código e instruções (concluído); backlog com requisitos, regras de negócio e critérios de aceitação (concluído); protótipos e design system no Figma (produzidos pelo grupo); modelo do banco e scripts de criação e carga (concluído); diagramas e documentação da arquitetura e das classes (produzidos pelo grupo e descritos neste relatório); aplicação web responsiva com os três módulos e o fluxo completo (concluída); plano e evidências de testes (concluído); sistema implantado com dados fictícios (concluído); e este relatório final.

15. Limitações e trabalhos futuros

15.1 Limitações conhecidas

- **Envio de e-mail simulado:** a recuperação de senha gera o link corretamente, mas o e-mail é exibido no console do servidor em vez de ser enviado. Em produção, o link precisa ser obtido nos registros do servidor.
- **Documentação da API:** não foi criada a documentação interativa em OpenAPI/Swagger; as rotas estão descritas no README, neste relatório e nos documentos de entrega.
- **Testes automatizados permanentes:** os testes automatizados ponta a ponta foram executados durante o desenvolvimento, mas não ficaram integrados ao repositório como suíte de testes de execução contínua.
- **Backup:** o banco em produção depende do serviço gerenciado do Railway; não foi formalizada uma rotina própria de backup e restauração.
- **Ano letivo fixo na interface:** as telas usam o ano letivo de 2026 como padrão.
- **Plano gratuito de hospedagem:** o Railway em período de avaliação tem limites de uso e de tempo, adequados para a demonstração, mas não para uso contínuo.
- **Extensão opcional:** o dashboard com evolução das médias ao longo do ano e a agenda de reuniões, previstos como extensão, não foram implementados. Os relatórios de médias e de tags cobrem parte dessa necessidade.

15.2 Trabalhos futuros

- Integrar um serviço real de e-mail para a recuperação de senha e para avisar as famílias quando um relatório for publicado.
- Documentar a API em Swagger e criar testes automatizados de API e de interface executados a cada commit (integração contínua).
- Implementar a extensão: dashboard com gráficos da evolução das médias e da distribuição de tags por bimestre, e agenda de reuniões entre responsáveis e professores.
- Permitir a escolha do ano letivo na interface e a virada de ano com cópia da estrutura de turmas.
- Criar rotina de backup diário do banco com teste de restauração.

- Adicionar termo de consentimento, política de privacidade e funcionalidades de atendimento aos direitos do titular, para adequação completa à LGPD.
- Oferecer notificações no navegador e um aplicativo instalável (PWA) para as famílias.

16. Lições aprendidas

- **Commitar cedo e sempre.** A perda de trabalho na troca de máquina mostrou que o código só está seguro quando está no repositório.
- **Validar o mais difícil primeiro.** Começar pelo ambiente, pela conexão com o banco, pela autenticação e pela arquitetura reduziu os riscos e deixou o restante mais previsível.
- **Blocos pequenos e testáveis.** Implementar um bloco por vez, testar e só então seguir evitou acumular erros difíceis de localizar.
- **Ler a mensagem de erro inteira.** A maior parte dos problemas foi resolvida lendo com atenção o terminal e o console, em vez de tentar soluções às cegas.
- **Regras no servidor, não na tela.** A interface ajuda o usuário, mas a segurança e as regras de negócio precisam estar na API e no banco.
- **Segurança desde o início.** Senhas em hash, consultas parametrizadas e segredos fora do código são mais fáceis de fazer certo desde o começo do que de corrigir depois.
- **Organização do repositório é trabalho de equipe.** Combinar onde fica cada coisa evita que uma entrega de disciplina quebre o sistema.
- **Documentar também é entregar.** Deixar a documentação para o final concentrou muito esforço nos últimos dias; documentar aos poucos teria distribuído melhor o trabalho.
- **A inteligência artificial ajuda, mas o entendimento é nosso.** Usar o assistente de forma guiada, digitando, testando e entendendo cada passo, foi o que permitiu à equipe explicar, manter e corrigir o próprio sistema.

17. Conclusão

O projeto atingiu o objetivo geral: a KFKA – Plataforma de Acompanhamento Escolar é uma aplicação web responsiva, segura e integrada que permite registrar, revisar, publicar e acompanhar as informações acadêmicas bimestrais de alunos do Ensino Fundamental. O professor tem um espaço simples para registrar o desenvolvimento de cada aluno dentro do prazo; a escola controla o que é publicado, com histórico de todas as alterações; e as famílias acompanham os filhos de forma clara, pelo celular, com ciência, retorno e PDF.

Todos os objetivos específicos foram cumpridos, e o cenário de ponta a ponta exigido pelo enunciado funciona no sistema publicado. Do ponto de vista técnico, o projeto reuniu em um só produto o conteúdo das disciplinas do semestre: a API e a interface de Desenvolvimento Web Full Stack, o modelo relacional e os índices de Banco de Dados, as classes e a injeção de dependência de Programação Orientada a Objetos, as estruturas de Estrutura de Dados e o guia de estilo e os protótipos de Design de Interface Digital.

Para a equipe, o maior ganho foi passar por todo o ciclo de um software real: entender um problema, decidir a tecnologia, montar o ambiente, modelar, construir, testar, publicar e documentar, enfrentando e resolvendo problemas concretos em cada etapa. O sistema está pronto para demonstração e oferece uma base sólida para a evolução descrita nos trabalhos futuros.

Referências

BRASIL. **Lei nº 13.709, de 14 de agosto de 2018.** Lei Geral de Proteção de Dados Pessoais (LGPD). Brasília, 2018. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm.

EXPRESS. **Express 5 – documentação oficial.** Disponível em: https://expressjs.com.

MDN WEB DOCS. **HTTP response status codes.** Mozilla. Disponível em: https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Status.

MYSQL. **MySQL 8.0 Reference Manual.** Oracle. Disponível em: https://dev.mysql.com/doc/refman/8.0/en/.

NODE.JS. **Documentação do Node.js.** OpenJS Foundation. Disponível em: https://nodejs.org/docs.

OWASP FOUNDATION. **OWASP Top 10 – riscos de segurança em aplicações web.** Disponível em: https://owasp.org/www-project-top-ten/.

PDFKIT. **Documentação do PDFKit.** Disponível em: https://pdfkit.org.

RAILWAY. **Railway Docs.** Disponível em: https://docs.railway.com.

REACT. **React – documentação oficial.** Disponível em: https://react.dev.

REACT ROUTER. **React Router – documentação.** Disponível em: https://reactrouter.com.

JONES, M.; BRADLEY, J.; SAKIMURA, N. **RFC 7519: JSON Web Token (JWT).** IETF, 2015. Disponível em: https://datatracker.ietf.org/doc/html/rfc7519.

VITE. **Vite – documentação oficial.** Disponível em: https://vite.dev.

Apêndice A – Acessos de demonstração

O sistema publicado em https://projeto12-production.up.railway.app usa apenas dados fictícios. Os acessos de demonstração são:

- **Administrador:** admin@escola.com, senha Admin2026.
- **Professores:** beatriz@escola.com ou carlos@escola.com, senha Prof2026.
- **Responsável:** maria@email.com (mãe de Lucas e Júlia), senha Resp2026.

Nos dados de demonstração, o 1º, o 2º e o 3º bimestres de 2026 estão encerrados e publicados, e o 4º está aberto para digitação, o que permite demonstrar todo o fluxo.

Roteiro sugerido de demonstração

1. Abrir a página inicial e entrar como administrador; mostrar o painel, os cadastros, uma turma com alunos e disciplinas, e os bimestres.
2. Sair e entrar como a professora Beatriz; abrir uma turma no 4º bimestre, lançar descrição, média e tags de um aluno, salvar o rascunho e enviar para revisão.
3. Voltar ao administrador; abrir a revisão, mostrar a linha do tempo, alterar algo, devolver ou publicar.
4. Entrar como a responsável Maria; abrir o relatório do filho, registrar ciência com uma observação e baixar o PDF.
5. Voltar ao administrador; abrir os relatórios e exportar para Excel.
6. Mostrar o sistema no celular.

Apêndice B – Rotas da API

Autenticação

```
POST /auth/login                  login (público)
GET  /auth/me                     usuário logado
POST /auth/recuperar-senha        pede o link de recuperação (público)
POST /auth/redefinir-senha        define nova senha com o link (público)
POST /auth/alterar-senha          troca a senha do usuário logado
```

Cadastros (administrador)

```
GET | POST         /alunos   /professores   /responsaveis
PUT | DELETE       /alunos/:id   /professores/:id   /responsaveis/:id
GET | POST         /areas   /disciplinas   /turmas   /tags
PUT | DELETE       /areas/:id   /disciplinas/:id   /turmas/:id   /tags/:id
```

Vínculos (administrador)

```
GET | POST         /turmas/:id/alunos
DELETE             /turmas/:id/alunos/:alunoId
GET | POST         /turmas/:id/disciplinas
PUT | DELETE       /turmas/:id/disciplinas/:disciplinaId
GET | POST         /responsaveis/:id/alunos
DELETE             /responsaveis/:id/alunos/:alunoId
```

Bimestres

```
GET  /bimestres?anoLetivo=2026            administrador, professor e responsável
PUT  /bimestres/:ano/:numero              administrador
```

Professor

```
GET  /professor/turmas
GET  /professor/retornos
GET  /professor/turmas/:t/disciplinas/:d/bimestres/:n
GET  /professor/turmas/:t/disciplinas/:d/bimestres/:n/alunos/:a
PUT  /professor/turmas/:t/disciplinas/:d/bimestres/:n/alunos/:a
POST /professor/acompanhamentos/:id/enviar
```

Revisão (administrador)

```
GET  /revisao/acompanhamentos
GET  /revisao/acompanhamentos/:id
PUT  /revisao/acompanhamentos/:id
POST /revisao/acompanhamentos/:id/devolver
POST /revisao/acompanhamentos/:id/publicar
```

Responsável

```
GET  /responsavel/alunos
GET  /responsavel/alunos/:id/relatorio?anoLetivo=2026&bimestre=n
GET  /responsavel/alunos/:id/relatorio/pdf?anoLetivo=2026&bimestre=n
POST /responsavel/alunos/:id/ciencia
```

Relatórios (administrador)

```
GET  /relatorios/medias
GET  /relatorios/status
GET  /relatorios/tags
GET  /relatorios/retornos
GET  /relatorios/exportar?formato=xlsx | csv
```

Apêndice C – Instalação e execução local

Pré-requisitos: Node.js 20 ou superior, MySQL 8 e Git. No Windows, caso o PowerShell bloqueie o comando npm, use `npm.cmd`.

1. Clonar o repositório: `git clone https://github.com/2026-2-NADS2/Projeto12.git`.
2. Ligar o MySQL.
3. Na pasta `backend`, instalar as dependências com `npm install`.
4. Copiar `backend/.env.example` para `backend/.env` e preencher a senha do MySQL e uma chave JWT.
5. Criar o banco com os dados de demonstração: `npm run dados`.
6. Subir a API: `npm start` (endereço http://localhost:3001).
7. Em outro terminal, na pasta `frontend`, executar `npm install` e `npm run dev`.
8. Abrir http://localhost:5173 no navegador e entrar com um dos acessos do Apêndice A.

   Variáveis de ambiente do backend:

```
DB_HOST=127.0.0.1
DB_PORT=3306
DB_USER=root
DB_PASSWORD=sua_senha_do_mysql
DB_NAME=sistema_escolar
JWT_SECRET=troque_por_um_texto_longo_e_aleatorio
FRONTEND_URL=http://localhost:5173
```

Apêndice D – Glossário

- **Acompanhamento:** registro bimestral de um aluno em uma disciplina, com descrição, média e tags.
- **Bimestre:** cada uma das quatro divisões do ano letivo, com período de digitação definido pelo administrador.
- **Ciência:** confirmação do responsável de que leu o relatório do bimestre.
- **Exclusão lógica:** marcar um registro como inativo em vez de apagá-lo, preservando o histórico.
- **Fluxo editorial:** caminho do acompanhamento entre rascunho, envio, revisão, devolução e publicação.
- **Hash:** transformação de um texto, como a senha, em um código que não pode ser revertido.
- **Middleware:** função executada antes da rota, usada para autenticar, autorizar ou validar.
- **Pool de conexões:** conjunto de conexões com o banco reaproveitadas entre as requisições.
- **Repositório:** classe que concentra o acesso ao banco de uma entidade.
- **Tag:** etiqueta cadastrada pelo administrador para classificar o acompanhamento (por exemplo, "Participativo").
- **Token JWT:** credencial assinada pelo servidor que identifica o usuário nas requisições.
- **Vínculo:** relação entre turma e aluno, turma e disciplina com professor, ou responsável e aluno.